SCIENTIFIC COMPUTING SOLVERS

Ecuații și sisteme neliniare

Teorie, demonstrații esențiale și exemple rezolvate

Partea I a proiectului

Pentru Maria Alexandra Timbuș

Material de studiu în limba română, redactat în LaTeX.

Bibliografie de bază: Michael T. Heath, capitolul 5;

Endre Süli și David F. Mayers, capitolele 1 și 4.

5 octombrie 2026

Cuprins

1 Scopul acestei etape și ordinea de studiu 2

2 Problema matematică: rădăcini, existență și unicitate 2

3 Eroare, reziduu și condiționare 3

4 Convergența unei metode iterative 3

5 Metoda bisecției 4

6 Iterația de punct fix și contracțiile 5

7 Metoda Newton pentru ecuații scalare 6

8 Metoda secantei 7

9 Metode hibride și alte metode utile 8

10 Oprire corectă și aritmetică floating-point 9

11 Sisteme neliniare și Jacobianul 10

12 Newton pentru sisteme 10

13 Cum obții Jacobianul și ce este Broyden? 11

14 Compararea metodelor și protocolul de benchmark 12

15 Planul de învățare și verificarea cunoștințelor 13

16 Fișă de formule și bibliografie 15

1 Scopul acestei etape și ordinea de studiu

Scopul este să înțelegi cum se construiesc, analizează și verifică metodele numerice pentru $f(x) = 0$ și $F(x) = 0$. Materialul acoperă fundația necesară pentru implementările din prima etapă; FDM, PINN și accelerarea pe GPU apar ulterior.

Ordinea recomandată este: problema și condiționarea, convergența, bisecția, punctul fix, Newton, secanta, metodele hibride, apoi sistemele neliniare. Înainte să compari viteza, verifică dacă metodele rezolvă problema corect și ating o precizie comparabilă.

Prerechizite. Continuitate, derivată, teorema valorilor intermediare, teorema valorii medii, Taylor, vectori, norme și rezolvarea unui sistem liniar. Formulele necesare sunt reluate aici.

Ce păstrezi în repository. În `theory/` pui acest PDF și sursa LaTeX. În `exercises/` pui enunțurile, calculele și experimentele tale. În `src/` pui solverele reutilizabile. Nu include copiile integrale ale cărților într-un repository public fără drept de distribuire.

2 Problema matematică: rădăcini, existență și unicitate

2.1 Ce rezolvăm?

O ecuație scalară este scrisă în forma

$$f(x) = 0, \qquad f : I \subseteq \mathbb{R} \longrightarrow \mathbb{R}.$$

O rădăcină α verifică $f(\alpha) = 0$. De exemplu, $e^{-x} = x$ devine $f(x) = e^{-x} - x = 0$. Nu toate ecuațiile au soluții reale; unele au mai multe sau infinit de multe.

Teoremă 2.1 (Existență prin schimbare de semn)**.** *Dacă f este continuă pe $[a, b]$ și $f(a)f(b) < 0$, atunci există cel puțin o rădăcină în (a, b).*

Condiția nu garantează unicitatea. Dacă f este strict monotonă pe interval, rădăcina este unică. Un criteriu suficient pentru monotonie strictă este $f'(x) > 0$ peste tot sau $f'(x) < 0$ peste tot în interior.

Exemplu. Pentru $f(x) = e^{-x} - x$, $f(0) = 1 > 0$, $f(1) = e^{-1} - 1 < 0$, iar $f'(x) = -e^{-x} - 1 < 0$. Există exact o rădăcină în $[0, 1]$.

De reținut. Schimbarea de semn fără continuitate nu este suficientă: $f(x) = 1/x$ are semne opuse la -1 și 1, dar nu are rădăcină. Lipsa schimbării de semn nu exclude o rădăcină: $(x - 1)^2$ se anulează fără să schimbe semnul.

2.2 Rădăcini simple și multiple

Rădăcina este simplă dacă $f(\alpha) = 0$ și $f'(\alpha) \neq 0$. Pentru o funcție suficient de netedă, multiplicitatea m este definită prin

$$f(\alpha) = \cdots = f^{(m-1)}(\alpha) = 0, \quad f^{(m)}(\alpha) \neq 0.$$

Local, $f(x) = (x - \alpha)^m h(x)$ cu $h(\alpha) \neq 0$. Pentru m par nu apare schimbare de semn suficient de aproape de rădăcină; pentru m impar apare. Condiția $f'(\alpha) = 0$ indică o rădăcină nesimplă, dar singură nu stabilește o multiplicitate finită pentru orice funcție netedă.

3 Eroare, reziduu și condiționare

3.1 Trei mărimi diferite

Pentru o aproximație x_k, definim

$$e_k = |x_k - \alpha|, \qquad r_k = |f(x_k)|, \qquad s_k = |x_k - x_{k-1}|.$$

Eroarea măsoară distanța față de soluție, reziduul cât de bine este satisfăcută ecuația, iar pasul cât s-a schimbat aproximația. Într-o problemă reală α este necunoscută, deci e_k nu este disponibil direct.

Dacă între x și α avem $|f'(t)| \geq m > 0$, teorema valorii medii dă

$$|x - \alpha| \leq \frac{|f(x)|}{m}.$$

Aceasta este o garanție; aproximația $e_k \approx |f(x_k)/f'(\alpha)|$ este doar locală.

Exemplu de reziduu înșelător. Pentru $f(x) = 10^{-12}(x-1)$ și $x = 2$, reziduul este 10^{-12}, dar eroarea este 1. Pentru $f(x) = (x-1)^4$, un reziduu 10^{-12} corespunde erorii 10^{-3}.

3.2 Sensibilitatea rădăcinii

Perturbăm ecuația la $f(x) + \delta = 0$. Lângă o rădăcină simplă,

$$f'(\alpha)\Delta\alpha + \delta \approx 0, \quad \Delta\alpha \approx -\frac{\delta}{f'(\alpha)}.$$

Pentru această perturbare aditivă, sensibilitatea absolută locală este $1/|f'(\alpha)|$. Ea depinde de modelul perturbării și de scalarea funcției; nu este o măsură universală independentă de reprezentare. Rădăcinile multiple sunt sensibile: la $x^2 - \delta = 0$ deplasarea este de ordin $\sqrt{\delta}$.

De reținut. Un algoritm bun nu elimină condiționarea slabă a problemei. Raportează atât reziduul, cât și o estimare sau o măsurare a erorii.

4 Convergența unei metode iterative

4.1 Definiții și ordine

O metodă produce $x_0, x_1, \ldots$. Convergența înseamnă $x_k \to \alpha$. Pentru $e_k > 0$, ordinul asimptotic p este caracterizat prin

$$\lim_{k\to\infty} \frac{e_{k+1}}{e_k^p} = C, \qquad 0 < C < \infty.$$

Pentru convergență liniară cerem $p = 1$ și $0 < C < 1$. Convergența superliniară înseamnă $e_{k+1}/e_k \to 0$; ordinul $p > 1$ este un caz particular. Convergența pătratică are $p = 2$.

La convergența liniară se câștigă aproximativ un număr constant de cifre corecte pe pas. La convergența pătratică, suficient de aproape de soluție, numărul cifrelor corecte se dublează aproximativ.

4.2 Locală versus globală

O garanție locală cere ca x_0 să fie suficient de aproape de soluție. O garanție pe un interval cere ipoteze valabile pe întregul interval. Newton și secanta sunt în general metode cu garanții locale. Bisecția converge din orice interval valid cu schimbare de semn și continuitate.

4.3 Estimarea experimentală a ordinului

Dacă ai o soluție de referință suficient de precisă,

$$\widehat{p}_k = \frac{\log(e_{k+1}/e_k)}{\log(e_k/e_{k-1})}.$$

Folosește formula numai în regimul asimptotic, cu erori pozitive și distincte, înainte de plafonul de rotunjire. Fără soluție exactă, poți înlocui erorile cu pașii succesivi; aceasta este o estimare euristică, sensibilă la oscilații și stagnare.

Bisecția cere atenție. Limita superioară a erorii se înjumătățește; erorile efective ale mijloacelor nu trebuie să aibă raportul $1/2$ și nici să descrească monoton.

5 Metoda bisecției

5.1 Algoritm și invariant

Verificăm continuitatea matematic și evaluăm capetele. Dacă unul este rădăcină, îl returnăm. Altfel, avem $f(a)f(b) < 0$. Calculăm

$$c = a + \frac{b-a}{2}.$$

Dacă $f(c) = 0$, încheiem; altfel păstrăm jumătatea cu semne opuse la capete. Invariantul este că intervalul curent conține o rădăcină și păstrează schimbarea de semn.

În implementare comparăm semnele, evitând produsul $f(a)f(c)$ care poate produce overflow sau underflow. Pentru capete finite extreme, și formula mijlocului trebuie protejată de overflow; $a + (b-a)/2$ nu este sigură pentru orice pereche de numere floating-point.

5.2 Garanția de eroare și numărul de pași

Notăm intervalul inițial $I_0 = [a_0, b_0]$, de lungime L_0. După n înjumătățiri, $|I_n| = L_0/2^n$. Mijlocul c_n al lui I_n verifică

$$|c_n - \alpha| \le \frac{L_0}{2^{n+1}}.$$

Pentru eroare cel mult ε, sunt suficiente

$$n \ge \max\left(0, \left\lceil \log_2 \frac{L_0}{2\varepsilon} \right\rceil\right)$$

înjumătățiri urmate de evaluarea mijlocului. Indexarea contează: aici c_0 este mijlocul intervalului inițial.

5.3 Exemplu calculat

Pentru $f(x) = x^3 - 2x - 5$, $f(2) = -1$, $f(3) = 16$.

a	b	c	$f(c)$
2	3	2.5	5.625
2	2.5	2.25	1.890625
2	2.25	2.125	0.345703125
2	2.125	2.0625	−0.351318359

După ultima decizie, rădăcina este în $[2.0625, 2.125]$. Pentru eroare 10^{-6} pornind din $[2, 3]$ sunt suficiente 19 înjumătățiri și mijlocul intervalului rezultat.

Cost. Două evaluări inițiale și o evaluare nouă a funcției pe pas, cu memorie constantă dacă nu stochezi istoricul. Este robustă, dar nu exploatează forma locală a funcției. Nu detectează direct rădăcini pare fără un bracket valid.

6 Iterația de punct fix și contracțiile

6.1 Reformularea

Rescriem ecuația ca $x = g(x)$ și iterăm

$$x_{k+1} = g(x_k).$$

Verifică echivalența pe domeniul ales: împărțirea la x sau extragerea unei rădăcini pătrate poate exclude soluții ori selecta numai o ramură.

Teoremă 6.1 (Contracție pe un interval)**.** *Fie $g : [a, b] \to [a, b]$ și $|g(x) - g(y)| \le q|x - y|$ pentru toate x, y, cu $0 \le q < 1$. Atunci există un punct fix unic în interval, iar iterația converge la el pentru orice $x_0 \in [a, b]$.*

O condiție suficientă este $g \in C^1([a, b])$ și $\sup |g'| \le q < 1$, împreună cu $g([a, b]) \subseteq [a, b]$.

Ideea demonstrației. Diferențele succesive sunt limitate de o progresie geometrică: $|x_{k+1} - x_k| \le q^k |x_1 - x_0|$. Suma cozilor tinde la zero, deci șirul este Cauchy și converge în intervalul închis. Continuitatea dă $\alpha = g(\alpha)$. Două puncte fixe ar satisface $|\alpha - \beta| \le q|\alpha - \beta|$, deci sunt egale.

6.2 Limite de eroare

Avem estimarea a priori

$$|x_k - \alpha| \le \frac{q^k}{1 - q} |x_1 - x_0|,$$

și, pentru $k \ge 1$, estimarea a posteriori

$$|x_k - \alpha| \le \frac{q}{1 - q} |x_k - x_{k-1}|.$$

Aceste formule transformă pasul într-o garanție numai dacă q este justificat pe domeniul în care iterezi.

6.3 Analiză locală

Taylor dă $x_{k+1} - \alpha = g'(\alpha)(x_k - \alpha) + O((x_k - \alpha)^2)$. Dacă $0 < |g'(\alpha)| < 1$, convergența este local liniară. Dacă $|g'(\alpha)| > 1$, punctul fix este local repulsiv. La $|g'(\alpha)| = 1$, criteriul nu decide. Dacă $g'(\alpha) = 0$ și g este suficient de netedă, poate apărea ordin cel puțin doi.

6.4 Aceeași ecuație, trei comportamente

Pentru $x^2 - x - 2 = 0$, rădăcinile sunt 2 și -1. La rădăcina 2:

$g(x)$	$g'(2)$	comportament local
$x^2 - 2$	4	repulsiv
$\sqrt{x+2}$	1/4	atractiv
$1 + 2/x$	$-1/2$	atractiv, cu alternanță

Forma cu radical nu păstrează rădăcina negativă. Forma $1 + 2/x$ exclude $x = 0$.

Pentru $e^{-x} = x$, $g(x) = e^{-x}$. Pe $[0.5, 1]$, g ia valori în $[e^{-1}, e^{-0.5}]$, deci intervalul nu este invariant. În schimb $[0.3, 0.8]$ este invariant și $q = e^{-0.3} < 1$. Aceasta arată de ce trebuie verificate ambele ipoteze, nu numai derivata.

6.5 Relaxare

Se poate folosi $g_\omega(x) = (1 - \omega)x + \omega g(x)$. Derivata locală este $1 - \omega + \omega g'(\alpha)$. Alegerea lui ω poate stabiliza sau accelera anumite probleme; nu există o valoare universală. O altă formă este $g(x) = x - \lambda f(x)$, pentru care condiția locală este $|1 - \lambda f'(\alpha)| < 1$.

7 Metoda Newton pentru ecuații scalare

7.1 Deducerea formulei

Aproximăm f prin tangenta în x_k:

$$f(x_k + s) \approx f(x_k) + f'(x_k)s.$$

Impunem anularea modelului și obținem

$$s_k = -\frac{f(x_k)}{f'(x_k)}, \qquad x_{k+1} = x_k + s_k.$$

Este o liniarizare locală. Nu căutăm minimul lui f; căutăm un zero.

7.2 Convergență pătratică și demonstrație

Dacă f este de clasă C^2 lângă o rădăcină simplă și x_0 este suficient de aproape, Newton converge cel puțin pătratic. Scriem, cu ξ_k între x_k și α,

$$0 = f(\alpha) = f(x_k) + f'(x_k)(\alpha - x_k) + \frac{1}{2}f''(\xi_k)(\alpha - x_k)^2.$$

Combinând cu formula Newton,

$$x_{k+1} - \alpha = \frac{f''(\xi_k)}{2f'(x_k)}(x_k - \alpha)^2.$$

Derivata rămâne separată de zero local, deci $e_{k+1} \le Ce_k^2$. Dacă $f''(\alpha) \neq 0$, constanta asimptotică este $|f''(\alpha)/(2f'(\alpha))|$. Dacă aceasta este zero, ordinul poate fi mai mare sau metoda poate termina exact.

7.3 Exemplu: $\sqrt{2}$

Pentru $f(x) = x^2 - 2$, formula devine

$$x_{k+1} = \frac{1}{2}\left(x_k + \frac{2}{x_k}\right).$$

Cu $x_0 = 1$, obținem $x_1 = 1.5$, $x_2 = 1.4166666667$, $x_3 \approx 1.4142156863$, $x_4 \approx 1.4142135624$. Comportamentul rapid apare după intrarea în vecinătatea soluției.

7.4 Rădăcini multiple și Newton modificat

Dacă $f(x) = (x-\alpha)^m h(x)$, $m > 1$, Newton standard are, în condițiile uzuale, convergență liniară cu factor $(m-1)/m$. Pentru $f(x) = (x-1)^4$:

$$x_{k+1} - 1 = \frac{3}{4}(x_k - 1).$$

Cu $x_0 = 10$, $x_k = 1 + 9(3/4)^k$. Dacă multiplicitatea m este cunoscută, formula

$$x_{k+1} = x_k - m\frac{f(x_k)}{f'(x_k)}$$

restabilește în mod obișnuit convergența pătratică; pentru puterea pură de mai sus ajunge exact la rădăcină într-un pas, în aritmetică exactă.

7.5 Cum poate eșua?

Pentru $f(x) = x^3 - 2x + 2$, Newton din $x_0 = 0$ dă $x_1 = 1$, apoi $x_2 = 0$: apare un ciclu și nu există convergență. Pentru $x^3 - 1 = 0$ cu $x_0 = 0$, derivata este zero și pasul nu este definit.

Alte riscuri: un pas părăsește domeniul, funcția produce valori nefinite, derivata foarte mică produce pași enormi, metoda ajunge la altă rădăcină. Pragul pentru derivată trebuie raportat la scară; o constantă absolută aleasă arbitrar poate respinge o ecuație corectă dar scalată.

7.6 Newton amortizat

Folosim $x_{k+1} = x_k + \lambda_k s_k$, cu $0 < \lambda_k \le 1$. Alegem pasul prin reducere repetată până respectă domeniul și o condiție de scădere pentru $\phi(x) = \frac{1}{2}f(x)^2$. Dacă $f'(x_k) \neq 0$, direcția Newton satisface $\phi'(x_k)s_k = -f(x_k)^2$. Amortizarea îmbunătățește robustețea, dar nu garantează găsirea unei rădăcini în orice problemă.

8 Metoda secantei

8.1 Formulă și cost

Înlocuim derivata prin panta dintre ultimele două puncte:

$$x_{k+1} = x_k - f(x_k)\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Avem nevoie de două aproximații inițiale. După evaluările inițiale, reutilizăm valorile vechi, deci fiecare pas cere o singură evaluare nouă a lui f. Secanta nu păstrează automat un interval cu schimbare de semn.

8.2 De unde vine ordinul 1.618?

Lângă o rădăcină simplă, în cazul generic neted cu termenul dominant nenul, erorile semnate satisfac aproximativ

$$\varepsilon_{k+1} \approx \frac{f''(\alpha)}{2f'(\alpha)}\varepsilon_k\varepsilon_{k-1}.$$

Dacă ordinul este p, relația puterilor conduce la $p = 1 + 1/p$, deci

$$p = \frac{1+\sqrt{5}}{2} \approx 1.618.$$

Rezultatul este local, pentru porniri suficient de apropiate și condiții de regularitate; nu este o garanție pentru orice funcție sau rădăcină multiplă.

8.3 Exemplu și eșecuri

Pentru $f(x) = x^2 - 2$, cu $x_0 = 1$, $x_1 = 2$, obținem $x_2 = 4/3$, $x_3 = 1.4$, $x_4 \approx 1.4146341463$. Urmează convergența către $\sqrt{2}$.

Dacă $f(x_k) - f(x_{k-1}) = 0$, pasul nu este definit. O diferență foarte mică este sensibilă la rotunjire. Verifică domeniul, numerele finite și stagnarea. Dacă unul dintre puncte este deja rădăcină, oprește înainte de a calcula raportul.

9 Metode hibride și alte metode utile

9.1 Newton protejat prin bisecție

Păstrăm un bracket valid $[a, b]$. Încercăm un pas Newton; îl acceptăm numai dacă este în interval și satisface un criteriu de progres. Altfel folosim mijlocul. Actualizăm bracketul după semnul funcției. Pentru garanție, strategia trebuie să impună contracția intervalului; simpla acceptare a oricărui punct interior nu este suficientă.

9.2 Regula falsi și Brent

Regula falsi folosește intersecția secantei cu axa, dar păstrează bracketul. Un capăt poate rămâne aproape fix mult timp, ducând la progres lent. Variantele modificate reduc acest efect.

Metoda Brent combină bisecția, secanta și interpolarea pătratică inversă cu verificări de acceptare. Pentru ecuații scalare continue cu bracket valid este o metodă practică de referință. Nu este obligatoriu să o implementezi în prima etapă; este util să înțelegi de ce combină robustețea cu pași rapizi.

9.3 Aitken și metode cvasi-Newton

Pentru un șir de punct fix, accelerarea Aitken este

$$\widehat{x}_k = x_k - \frac{(x_{k+1} - x_k)^2}{x_{k+2} - 2x_{k+1} + x_k}.$$

Poate accelera convergența liniară în condiții potrivite, dar numitorul mic și costul suplimentar trebuie tratate. Pentru sisteme, metodele cvasi-Newton actualizează o aproximație a Jacobianului, evitând reconstruirea sa completă la fiecare pas.

10 Oprire corectă și aritmetică floating-point

10.1 Toleranțe absolute și relative

Un criteriu pentru pas este

$$|x_{k+1} - x_k| \leq \mathrm{atol}_x + \mathrm{rtol}_x |x_{k+1}|.$$

Pentru reziduu folosim o scară fixă $S_f > 0$ relevantă problemei:

$$|f(x_{k+1})| \leq \mathrm{atol}_f + \mathrm{rtol}_f S_f.$$

Toleranțele pentru x și f au sensuri și eventual unități diferite. Pentru sisteme cu componente de scări diferite, normalizează variabilele și ecuațiile ori folosește norme ponderate.

10.2 Ce declarăm drept succes?

Pentru bisecție, jumătatea lățimii intervalului oferă un certificat de eroare pe x, dacă bracketul este valid. Pentru punct fix, estimarea de contracție justificată oferă alt certificat. Pentru Newton și secantă, un reziduu acceptabil plus un pas mic reprezintă un criteriu practic; fără informații de condiționare, nu certifică singur eroarea față de rădăcină.

Dacă pasul este mic dar reziduul rămâne mare, raportează *stagnare*, nu convergență. O rădăcină evaluată exact ca zero poate fi acceptată imediat, dar această egalitate este în aritmetica utilizată.

10.3 Limite numerice

În binary64, precizia relativă a mașinii este aproximativ $2.22 \cdot 10^{-16}$. Nu te aștepta la 16 cifre corecte în orice problemă. Pot apărea anulări, overflow, underflow sau $x_{k+1} = x_k$ prin rotunjire.

Pe lângă toleranțe, ai nevoie de: număr maxim de pași, verificarea valorilor finite, tratarea numitorilor problematici, respectarea domeniului și motiv explicit de terminare. La bisecție, dacă mijlocul coincide cu un capăt, nu mai poți restrânge intervalul în precizia curentă.

11 Sisteme neliniare și Jacobianul

11.1 Formulare și norme

Problema este $F(x) = 0$, cu $F : \mathbb{R}^n \to \mathbb{R}^n$ și $x = (x_1, \ldots, x_n)^T$. Definim

$$F(x) = \begin{pmatrix} f_1(x) \\ \vdots \\ f_n(x) \end{pmatrix}, \qquad J_F(x) = \left[\frac{\partial f_i}{\partial x_j}(x) \right]_{i,j=1}^{n}.$$

Normele uzuale sunt $\|x\|_2 = (\sum_i x_i^2)^{1/2}$ și $\|x\|_\infty = \max_i |x_i|$. Alege o normă și păstreaz-o consecvent în rezultate.

Un Jacobian nesingular la soluție implică izolarea locală a soluției sub ipotezele uzuale de derivabilitate. Nu implică unicitate globală. Perturbațiile mici satisfac local $\Delta x \approx -J_F(\alpha)^{-1}\Delta F$; amplificarea este controlată de norma inversei, în normele și scările alese.

11.2 Punct fix vectorial

Scriem $x = G(x)$ și iterăm $x_{k+1} = G(x_k)$. Dacă G transformă o mulțime închisă completă D în ea însăși și este contracție pe D, există un punct fix unic și convergență pentru orice pornire din D.

Pe un domeniu convex, o limită uniformă $\|J_G(x)\| \leq q < 1$ într-o normă matricială indusă este o condiție suficientă. Local, $\rho(J_G(\alpha)) < 1$, împreună cu regularitatea necesară, asigură atractivitate; ρ este raza spectrală. Valoarea exact 1 nu decide comportamentul.

În iterația simultană toate componentele noi se calculează din vectorul vechi. Actualizarea succesivă folosește componentele deja actualizate și produce o altă metodă, cu altă analiză de convergență.

12 Newton pentru sisteme

12.1 Deducere și algoritm

Liniarizarea este $F(x_k + s) \approx F(x_k) + J_F(x_k)s$. La fiecare pas:

1. Evaluezi $F_k = F(x_k)$ și verifici oprirea.
2. Construiești $J_k = J_F(x_k)$.
3. Rezolvi sistemul liniar $J_k s_k = -F_k$.
4. Actualizezi $x_{k+1} = x_k + s_k$, eventual cu amortizare.
5. Verifici reziduul, pasul, domeniul și motivul terminării.

De reținut. Formula teoretică folosește J_k^{-1}, dar implementarea rezolvă sistemul liniar. Nu construiești explicit inversa.

12.2 Convergență și cost

Dacă F este suficient de netedă, Jacobianul este local Lipschitz și $J_F(\alpha)$ este nesingular, Newton are convergență locală pătratică din porniri suficient de apropiate, cu rezolvări liniare suficient de precise. Un Jacobian singular invalidează această garanție, fără să excludă orice convergență posibilă.

Pentru un sistem dens general, stocarea Jacobianului este $O(n^2)$, iar o factorizare directă obișnuită costă $O(n^3)$. Costul evaluării F și J se adaugă. Structura rară poate schimba radical costurile; nu converti automat o matrice rară în matrice densă.

12.3 Exemplu complet cu două ecuații

Considerăm

$$F(x,y) = \begin{pmatrix} x^2 + y^2 - 1 \\ x - y \end{pmatrix}, \qquad J_F(x,y) = \begin{pmatrix} 2x & 2y \\ 1 & -1 \end{pmatrix}.$$

Soluțiile sunt $(1/\sqrt{2}, 1/\sqrt{2})$ și $(-1/\sqrt{2}, -1/\sqrt{2})$. Din $(1,1)$, $F = (1,0)^T$, iar pasul rezolvă

$$\begin{cases} 2s_x + 2s_y = -1, \\ s_x - s_y = 0. \end{cases}$$

Rezultă $s_x = s_y = -1/4$, deci noul punct este $(0.75, 0.75)$. Următorul reziduu este $(0.125, 0)^T$, iar pasul este $(-1/24, -1/24)$, deci ajungem la $(0.7083333333, 0.7083333333)$.

Pornirea $(0,0)$ are Jacobian singular și sistemul Newton este incompatibil. La soluțiile de mai sus determinantul $-2(x+y)$ este nenul. Porniri diferite pot selecta soluții diferite.

12.4 Globalizare prin căutare liniară

Folosim $\phi(x) = \frac{1}{2}\,\|F(x)\|_2^2$. Pentru pasul Newton exact,

$$\nabla\phi(x_k)^T s_k = F_k^T J_k s_k = -\,\|F_k\|_2^2 < 0$$

când $F_k \neq 0$. O condiție Armijo este

$$\phi(x_k + \lambda s_k) \le \phi(x_k) - c\lambda\,\|F_k\|_2^2, \qquad 0 < c < 1.$$

Pornim cu $\lambda = 1$ și reducem, de exemplu înmulțind cu $1/2$. Sunt necesare limite și tratarea eșecului căutării. Globalizarea poate conduce la un punct staționar cu reziduu nenul; verifică reziduul final.

13 Cum obții Jacobianul și ce este Broyden?

13.1 Derivare analitică și diferențe finite

Pentru primele exerciții folosește derivate analitice și verifică-le independent. Aproximarea înainte pentru coloana j este

$$J_{:,j}(x) \approx \frac{F(x + h_j e_j) - F(x)}{h_j}.$$

Aproximarea centrată folosește $[F(x + h_j e_j) - F(x - h_j e_j)]/(2h_j)$. Eroarea de trunchiere este de ordin h_j, respectiv h_j^2, în ipotezele de netezime potrivite. Un pas prea mic amplifică rotunjirea și zgomotul.

Pentru evaluări netede și bine scalate, o regulă orientativă este $h_j \sim \sqrt{\epsilon_{\text{mach}}}\max(1, |x_j|)$ pentru diferențe înainte, respectiv $\epsilon_{\text{mach}}^{1/3}\max(1, |x_j|)$ pentru cele centrate. Nu sunt valori universal optime.

Automatic differentiation propagă derivate prin operațiile programului și evită eroarea de trunchiere a diferențelor finite, dar păstrează limitele aritmeticii și ale derivabilității operațiilor. PyTorch/JAX pot fi studiate după implementările de bază.

13.2 Actualizarea Broyden

Fie B_k o aproximație a Jacobianului. Rezolvăm $B_k s_k = -F(x_k)$, calculăm $y_k = F(x_{k+1}) - F(x_k)$ și actualizăm

$$B_{k+1} = B_k + \frac{(y_k - B_k s_k)s_k^T}{s_k^T s_k}.$$

Rezultă condiția secantei $B_{k+1}s_k = y_k$. Formula presupune $s_k \neq 0$. Metoda poate reduce costul reconstruirii Jacobianului, dar aproximația și sistemele liniare pot deveni problematice. Este o extensie, nu primul solver de implementat.

14 Compararea metodelor și protocolul de benchmark

Metodă	Date necesare	Convergență tipică	Limita principală
Bisecție	bracket valid	limita erorii se înjumătățește	cere continuitate și schimbare de semn
Punct fix	g, x_0	liniară la contracție cu derivată nenulă	reformularea poate diverge
Newton	f, f', x_0	pătratică local la rădăcină simplă	derivată mică, pornire nepotrivită
Secantă	f, x_0, x_1	ordin local generic 1.618	numitor mic, lipsă bracket
Newton sistem	F, J, x_0	pătratică local	cost liniar, singularitate

14.1 Ce măsori?

Pentru fiecare problemă raportează aproximația, eroarea față de referință când există, reziduul, iterațiile, evaluările lui f/F și ale derivatelor/Jacobianului, timpul și motivul terminării. Numărul de iterații singur nu arată costul: o iterație Newton poate fi mai scumpă decât o iterație secantă.

Folosește aceleași probleme și ținte de precizie. Pentru punct fix compară reziduul ecuației originale, nu numai $|g(x) - x|$. Aceste reziduuri pot avea scări diferite.

14.2 Măsurarea timpului și memoriei

Separă construirea datelor, timpul solverului și scrierea rezultatelor. Fă rulări repetate și raportează mediana și variabilitatea; pentru probleme mici, costul interpretorului poate domina. Stochează istoricul fie pentru toate metodele, fie pentru niciuna în măsurarea de performanță.

Memoria solverului scalar fără istoric este $O(1)$; cu k iterații stocate devine $O(k)$. Pentru sisteme dense, Jacobianul domină frecvent memoria. Specifică dacă măsori memorie Python, memorie totală a procesului sau memorie a dispozitivului; acestea nu sunt interschimbabile.

14.3 Set minim de probleme

1. $x^3 - 2x - 5 = 0$: problemă obișnuită; bracket $[2, 3]$.
2. $e^{-x} - x = 0$: rădăcină unică; testezi și punct fix.
3. $(x - 1)^4 = 0$: rădăcină multiplă; observi reziduul și Newton liniar.
4. $x^3 - 2x + 2 = 0$: testezi ciclul Newton din 0 și o metodă cu bracket $[-2, -1]$.
5. $10^{-12}(x - 1) = 0$: testezi scalarea reziduului.
6. Sistemul cerc–dreaptă din secțiunea 12: testezi două soluții și pornirea singulară.

Nu forța bisecția pe o problemă fără bracket. Marchează cazurile neaplicabile și eșecurile explicit, fără să le elimini din tabel.

15 Planul de învățare și verificarea cunoștințelor

15.1 Șase etape practice

1. Citește definițiile, existența, condiționarea și convergența. Explică diferența dintre eroare, reziduu și pas.
2. Implementează bisecția. Demonstrează limita de eroare și verifică numărul de pași.
3. Implementează punctul fix. Compară cele trei reformulări ale aceleiași ecuații și justifică intervalul invariant.
4. Implementează Newton și secanta. Reproduce convergența către $\sqrt{2}$, rădăcina multiplă și ciclul Newton.
5. Construiește benchmarkul scalar cu aceleași ținte de precizie, istorice și motive de terminare.
6. Implementează Newton pentru sisteme cu Jacobian analitic, rezolvare liniară și apoi amortizare.

15.2 Întrebări la care trebuie să poți răspunde

1. De ce schimbarea de semn cere continuitate? De ce nu certifică unicitatea?
2. Cum se obține limita erorii pentru bisecție?
3. De ce sunt necesare atât contracția, cât și păstrarea domeniului la punct fix?
4. Cum se deduce Newton din Taylor și unde se folosește $f'(\alpha) \neq 0$?

5. De ce Newton pierde convergența pătratică la rădăcini multiple?
6. De unde vine ordinul secantei și de ce nu înseamnă automat timp mai mic?
7. Cum poate reziduul să fie mic fără ca aproximația să fie bună?
8. De ce un pas mic poate însemna stagnare?
9. Ce reprezintă Jacobianul și de ce rezolvi un sistem în loc să calculezi inversa?
10. Ce îmbunătățește amortizarea și ce nu poate garanta?

15.3 Contractul comun pentru implementări

Rezultatul unui solver ar trebui să conțină `root`, `iterations`, `residual`, `converged`, `termination_reason`, numerele evaluărilor și opțional `history`. Timpul poate fi măsurat extern, pentru a păstra solverul ușor de testat. Istoricul păstrează punctul inițial, aproximațiile, reziduurile și pașii; pentru bisecție păstrează și bracketul.

În teste verifică atât succesul, cât și cazurile relevante de eșec: capăt rădăcină, bracket invalid, derivată zero, numitor secantă zero, domeniu invalid, valori nefinite, Jacobian singular, limită de iterații și stagnare. Nu scrie teste care doar repetă formula implementării; verifică proprietăți matematice și soluții cunoscute.

16 Fișă de formule și bibliografie

16.1 Formulele centrale

$$
\begin{aligned}
\text{Bisecție:} &\quad c = (a+b)/2, \quad |c-\alpha| \le (b-a)/2.\\
\text{Punct fix:} &\quad x_{k+1} = g(x_k), \quad \sup|g'| < 1 \text{ și interval invariant.}\\
\text{Newton:} &\quad x_{k+1} = x_k - f(x_k)/f'(x_k).\\
\text{Secantă:} &\quad x_{k+1} = x_k - f(x_k)\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.\\
\text{Newton sistem:} &\quad J_F(x_k)s_k = -F(x_k), \quad x_{k+1} = x_k + s_k.\\
\text{Ordin:} &\quad e_{k+1} \sim C e_k^p.\\
\text{Punct fix, certificat:} &\quad e_k \le \frac{q}{1-q}|x_k - x_{k-1}|.\\
\text{Reziduu–eroare:} &\quad e_k \le |f(x_k)|/m \quad \text{dacă } |f'| \ge m > 0.
\end{aligned}
$$

16.2 Unde citești în cărțile atașate

1. **Michael T. Heath, *Scientific Computing: An Introductory Survey.*** Capitolul 5, *Nonlinear Equations*: 5.1 problema și sensibilitatea; 5.2 metode scalare, convergență și metode de interpolare; 5.3 sisteme, punct fix, Newton și actualizări de tip secantă; 5.4 software. În exemplarul atașat capitolul începe la pagina tipărită 151. Folosește denumirile secțiunilor dacă numerotarea paginilor PDF diferă.
2. **Endre Süli și David F. Mayers, *An Introduction to Numerical Analysis.*** Capitolul 1, *Solution of equations in one variable*: iterație, convergență, relaxare, Newton, secantă și bisecție. Capitolul 4, *Simultaneous nonlinear equations*: iterație simultană, relaxare, Newton și convergență globală; începe la pagina tipărită 104.

Textul este o sinteză originală pentru studiu, nu o reproducere integrală a capitolelor. Exemplele și demonstrațiile dezvoltă conceptele de bază. Materialul include și observații practice de implementare și benchmark; pentru demonstrațiile extinse și exercițiile originale consultă bibliografia.

De reținut. Ordinea corectă a experimentului este: ipoteze matematice, implementare verificată, precizie comparabilă, apoi viteză și memorie.